

Transformer

摘要

主流的序列转换模型通常基于复杂的循环神经网络或卷积神经网络，这些模型结构包含编码器和解码器，而性能最佳的模型还会通过注意力机制连接编码器与解码器。本文提出了一种新的简单网络架构——Transformer，该模型完全基于注意力机制构建，彻底摒弃了循环结构和卷积结构。在两个机器翻译任务上的实验表明，Transformer 在翻译质量上优于现有模型，同时具有更强的并行计算能力，并且训练时间显著减少。在 WMT 2014 英语到德语翻译任务中，我们的模型在 WMT 2014 英德翻译任务中取得了 28.4 的 BLEU 分数，比现有的最佳结果，包括模型集成方法高出了 2 个以上的 BLEU 分数。；在 WMT 2014 英语到法语翻译任务中，该模型在仅使用 8 个 GPU 训练 3.5 天的情况下，取得了单模型最高水平的 41.8 BLEU 分数，而训练成本仅为已有最佳模型的一小部分。我们通过将 Transformer 成功应用于英语成分句法分析任务，证明了它在数据量充足和受限的情况下均能表现出色，从而表明该模型具有良好的泛化能力。

1. 引言

循环神经网络（Recurrent Neural Networks, RNN），尤其是长短期记忆网络（Long Short-Term Memory, LSTM）[13] 和门控循环神经网络（Gated Recurrent Neural Networks, GRU）[7]，已经成为序列建模和序列转换问题中的主流方法，例如语言模型和机器翻译任务 [35, 2, 5]。此后，大量研究工作不断推动循环语言模型以及编码器-解码器（encoder-decoder）架构的发展 [38, 24, 15]。

循环模型通常沿着输入序列和输出序列中的符号位置对计算过程进行分解。将序列中的位置与计算时间步数对应起来，它们会根据前一个隐藏状态 h_{t-1} 和当前位置 t 的输入，生成一系列隐藏状态 h_t 。这种天然的序列化计算方式，使得训练样本内部无法进行并行计算；而当序列长度较长时，这一问题变得尤为关键，因为内存限制会限制不同样本之间的批量处理规模。近年来，研究人员通过因式分解技巧（factorization tricks）[21] 和条件计算（conditional computation）[32]，在计算效率方面取得了显著提升，其中条件计算方法还进一步改善了模型性能。然而，序列计算所带来的根本限制仍然存在。

注意力机制（Attention mechanisms）已经成为许多序列建模和序列转换任务中重要的组成部分，它能够在不考虑输入序列或输出序列中位置距离的情况下，对不同位置之间的依赖关系进行建模 [2, 19]。然而，除了少数情况之外 [27]，这些注意力机制通常都是与循环网络结合使用的。

在本文中，我们提出了 Transformer，一种摒弃循环结构（recurrence）的模型架构，而是完全依赖注意力机制来建立输入和输出之间的全局依赖关系。Transformer 可以实现更高层次的并行计算，并且仅需在 8 块 P100 GPU 上训练 12 小时左右，就能够在机器翻译质量方面达到新的最先进水平。

2. 背景

减少序列计算的目标同样构成了 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础。这些模型都将卷积神经网络作为基本构建模块，可以并行地为所有输入位置和输出位置计算隐藏表示。在这些模型中，使任意两个输入或输出位置之间的信息建立关联所需的操作数量，会随着两个位置之间的距离增加而增长：对于 ConvS2S 来说是线性增长，而对于 ByteNet 来说则是对数增长。这使得模型更难学习远距离位置之间的依赖关系 [12]。在 Transformer 中，这一问题被降低到了常数数量的操作，尽管代价是由于对注意力权重位置进行平均而导致有效分辨率降低。不过，我们通过第 3.2 节所介绍的多头注意力机制（Multi-Head Attention）来抵消这种影响。

自注意力（Self-attention），有时也称为内部注意力（intra-attention），是一种通过关联同一序列中不同位置的信息来计算序列表征的注意力机制。自注意力已经成功应用于多种任务中，包括阅读理解、抽象式文本摘要、文本蕴含判断以及学习与具体任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络（End-to-end Memory Networks）采用了一种循环注意力机制，而不是基于序列位置对齐的循环结构，并且已经在简单语言问答任务和语言建模任务中取得了较好的效果 [34]。

据我们所知，Transformer 是第一个完全依赖自注意力机制来计算输入和输出表示，同时不使用基于序列对齐的循环神经网络（RNN）或卷积结构的序列转换模型。在接下来的章节中，我们将介绍 Transformer，阐述采用自注意力机制的动机，并讨论它相比于 [17]、[18] 和 [9] 等模型所具有的优势。

3. 模型架构

大多数具有竞争力的神经序列转换（neural sequence transduction）模型都采用编码器-解码器（encoder-decoder）结构 [5, 2, 35]。其中，编码器将输入的符号表示序列 $(x_1, \dots, x_n)$ 映射为一系列连续表示 $z = (z_1, \dots, z_n)$ 。在得到表示 z 后，解码器会逐个元素地生成输出符号序列 $(y_1, \dots, y_m)$ 。在每一步生成过程中，该模型都是自回归（auto-regressive）[10]，即在生成下一个元素时，会将之前已经生成的符号作为额外输入。

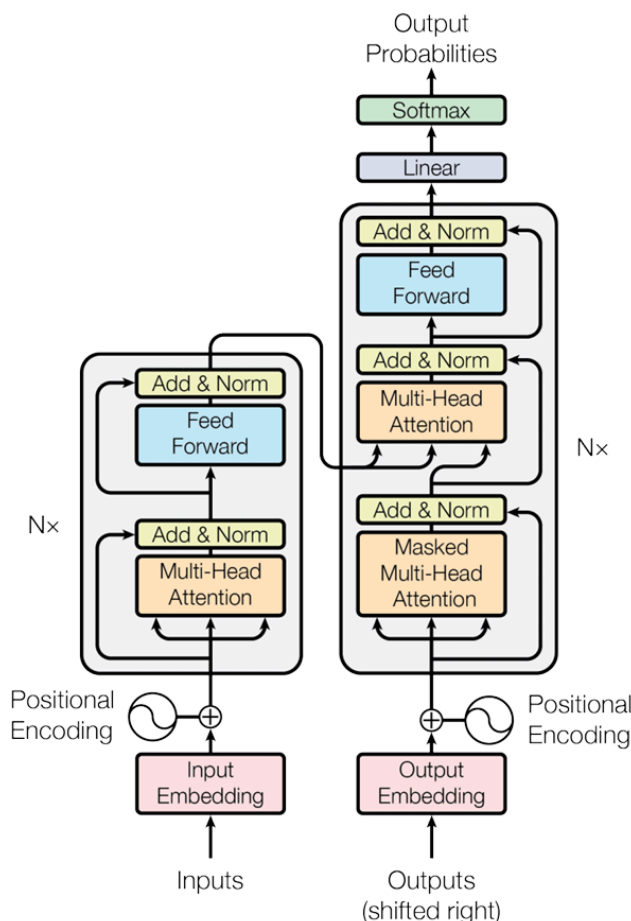


Figure 1: The Transformer - model architecture.

Transformer 遵循了这一整体架构，但在编码器和解码器中都采用了堆叠的自注意力层（stacked self-attention）以及逐点全连接层（point-wise, fully connected layers）作为基本组成模块，如图 1 的左半部分和右半部分分别所示。

3.1 编码器和解码器堆叠（Encoder and Decoder Stacks）

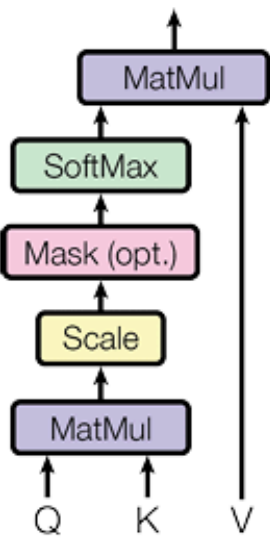
编码器（Encoder）：编码器由 $N = 6$ 个相同的层堆叠而成。每一层包含两个子层，第一个是多头自注意力机制（multi-head self-attention mechanism），第二个是简单的逐位置全连接前馈网络（position-wise fully connected feed-forward network）。我们在每个子层周围都采用残差连接（residual connection）[11]，并在之后进行层归一化（layer normalization）[1]。也就是说，每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 表示该子层本身所实现的函数。为了便于使用这些残差连接，模型中的所有子层以及嵌入层（embedding layers）的输出维度都设置为 $d_{\text{model}} = 512$ 。

解码器（Decoder）：解码器同样由 $N = 6$ 个相同的层堆叠而成。除了编码器每层中的两个子层之外，解码器还增加了第三个子层，该子层对编码器堆叠层的输出执行多头注意力操作。与编码器类似，我们在每个子层周围都采用残差连接，并随后进行层归一化。同时，我们修改了解码器堆叠中的自注意力子层，使其能够阻止当前位置关注后续位置的信息。这种掩码机制（masking）与输出嵌入向前偏移一个位置（offset by one position）相结合，确保位置 i 的预测只能依赖于位置 i 之前已经生成的已知输出。

3.2 注意力机制（Attention）

一个注意力函数（attention function）可以被描述为：将一个查询（query）和一组键-值对（key-value pairs）映射为一个输出，其中查询（query）、键（keys）、值（values）以及输出（output）都表示为向量。输出通过对值（values）进行加权求和得到，其中每个值所对应的权重，是通过查询（query）与其对应的键（key）之间的匹配度函数（compatibility function）计算得到的。

Scaled Dot-Product Attention



Multi-Head Attention

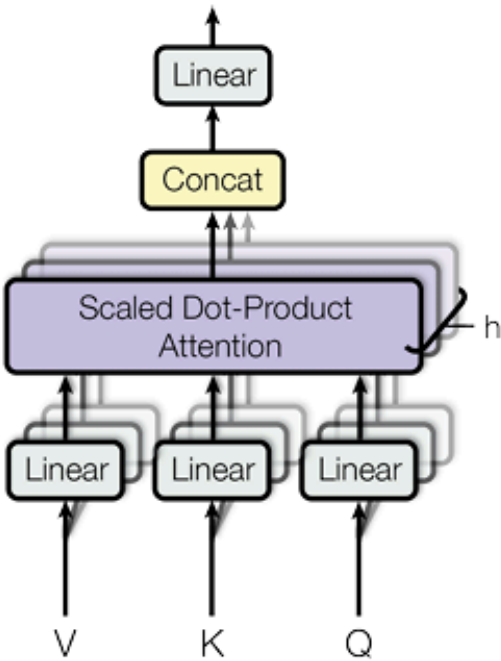


图2：（左）缩放点积注意力机制；（右）多头注意力机制由多个并行运行的注意力层组成。

3.2.1 缩放点积注意力（Scaled Dot-Product Attention）

我们将本文提出的特定注意力机制称为“缩放点积注意力”（如图2所示）。输入由维度为 d_k 的查询（queries）和键（keys），以及维度为 d_v 的值（values）组成。我们首先计算每个查询与所有键之间的点积，然后将每个点积结果除以 $\sqrt{d_k}$ ，最后通过 softmax 函数得到作用于各个值的权重。在实际应用中，我们会同时对一组查询计算注意力函数，并将这些查询组合成一个矩阵 Q 。同样，键和值也分别被组合成矩阵 K 和 V 。最终，我们通过以下公式计算输出矩阵：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

最常用的两种注意力函数是加性注意力（additive attention）[2] 和点积（乘法）注意力（dot-product / multiplicative attention）。点积注意力与我们提出的算法相同，唯一的区别是增加了缩放因子 $\frac{1}{\sqrt{d_k}}$ 。加性注意力通过一个带有单个隐藏层的前馈神经网络来计算兼容性函数。虽然从理论计算复杂度来看，两者较为相似，但在实际应用中，点积注意力的计算速度更快，并且更加节省空间，因为它可以通过高度优化的矩阵乘法代码实现。

当 d_k 较小时，两种注意力机制的性能表现相近；然而，当 d_k 较大时，加性注意力的效果优于未进行缩放的点积注意力 [3]。我们推测，对于较大的 d_k ，点积运算的结果会具有较大的数值幅度，使

softmax 函数进入梯度非常小的区域，从而导致梯度消失问题。为了抵消这一影响，我们将点积结果乘以缩放因子 $\frac{1}{\sqrt{d_k}}$ ，即对点积注意力进行缩放处理。

3.2.2 多头注意力 (Multi-Head Attention)

与仅使用一个具有 d_{model} 维度的键 (keys)、值 (values) 和查询 (queries) 进行单次注意力计算不同，我们发现，将查询、键和值分别通过 h 组不同的、可学习的线性投影，映射到维度分别为 d_k 、 d_k 和 d_v 的空间中是更加有效的。对于每一组经过投影后的查询、键和值，我们分别并行执行注意力函数计算，得到维度为 d_v 的输出结果。随后，将这些输出结果进行拼接 (concatenate)，并再次通过一次线性投影，得到最终的输出结果，如图2所示。

多头注意力机制使模型能够同时关注不同位置上的信息，并从不同的表示子空间 (representation subspaces) 中提取特征。相比之下，如果只使用单个注意力头，简单的平均操作会限制模型捕获多种不同信息表示的能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

其中：

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中这些投影矩阵为参数矩阵

$$W_i^Q \in \mathbb{R}^{d_{model} \times d_k}, W_i^K \in \mathbb{R}^{d_{model} \times d_k}, W_i^V \in \mathbb{R}^{d_{model} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_{model}}$$

在本文中，我们采用了 $h=8$ 个并行的注意力层，即 8 个注意力头 (heads)。对于每个注意力头，我们设置 $d_k = d_v = d_{model}/h = 64$ 由于每个注意力头的维度被降低，因此多头注意力的总体计算成本与使用完整维度的单头注意力机制相近。

3.2.3 注意力机制在本文模型中的应用

Transformer 在三种不同的方式中使用了多头注意力机制：

- 在“编码器-解码器注意力” (encoder-decoder attention) 层中，查询 (queries) 来自前一个解码器层，而键 (keys) 和值 (values) 来自编码器的输出。这样，解码器中的每一个位置都可以关注输入序列中的所有位置。这种机制与序列到序列 (sequence-to-sequence) 模型中常见的编码器-解码器注意力机制类似，例如文献 [38, 2, 9] 中所使用的方法。
- 编码器 (encoder) 包含自注意力 (self-attention) 层。在自注意力层中，所有的键 (keys)、值 (values) 和查询 (queries) 都来自同一个来源，在本文中即为编码器前一层的输出。通过这种方式，编码器中的每一个位置都可以关注编码器前一层中的所有位置。
- 类似地，解码器 (decoder) 中的自注意力层允许解码器中的每一个位置关注解码器中直到当前位置 (包括当前位置) 的所有位置。为了保持自回归 (auto-regressive) 特性，我们需要阻止解码器中的信息向左流动。因此，我们在缩放点积注意力 (scaled dot-product attention) 中通过掩码 (mask) 操作实现这一点：将 softmax 输入中所有对应于非法连接的位置设置为 $-\infty$ ，从而屏蔽这些位置的信息。具体过程如图2所示。

3.3 逐位置前馈网络 (Position-wise Feed-Forward Networks)

除了注意力子层（attention sub-layers）之外，我们的编码器和解码器中的每一层还包含一个全连接前馈网络（fully connected feed-forward network）。该网络会分别且相同地应用于序列中的每一个位置。该网络由两个线性变换（linear transformations）组成，并在两者之间加入一个 ReLU 激活函数，其计算公式为：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

虽然这些线性变换在不同位置之间共享相同的参数，但不同层之间使用不同的参数。另一种描述方式是，该网络可以看作两个卷积操作，其中卷积核大小（kernel size）为 1。该网络的输入维度和输出维度均为 $d_{model} = 512$ ，而内部隐藏层的维度为 $d_{ff} = 2048$ 。

3.4 嵌入（Embeddings）和 Softmax

与其他序列转换（sequence transduction）模型类似，我们使用可学习的嵌入（learned embeddings）将输入 token 和输出 token 转换为维度为 d_{model} 的向量。同时，我们使用常规的可学习线性变换（learned linear transformation）和 softmax 函数，将解码器的输出转换为预测下一个 token 的概率。

在我们的模型中，我们按照文献 [30] 的方法，在两个嵌入层以及 softmax 前的线性变换（pre-softmax linear transformation）之间共享相同的权重矩阵。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{model}}$ 。

表 1： 不同层类型的最大路径长度、每层计算复杂度以及最少顺序操作次数的比较。其中， n 表示序列长度， d 表示表示维度， k 表示卷积核大小， r 表示受限自注意力中的邻域大小。

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 位置编码（Positional Encoding）

由于我们的模型不包含循环结构（recurrence）和卷积操作（convolution），为了使模型能够利用序列中 token 的顺序信息，我们必须向模型中加入关于序列中 token 相对位置或绝对位置的信息。为此，我们在编码器和解码器堆栈的底部输入嵌入（input embeddings）中加入“位置编码”

（positional encodings）。位置编码与嵌入向量具有相同的维度 d_{model} ，因此二者可以直接相加。目前存在多种位置编码方式，包括可学习的位置编码和固定的位置编码 [9]。在本文中，我们采用不同频率的正弦函数和余弦函数：

$$\begin{aligned} PE(pos, 2i) &= \sin(pos/10000^{2i/d_{model}}) \\ PE(pos, 2i + 1) &= \cos(pos/10000^{2i/d_{model}}) \end{aligned}$$

其中， pos 表示位置， i 表示维度。也就是说，位置编码中的每一个维度都对应一个正弦函数。不同维度对应的波长（wavelength）构成了从 2π 到 $10000 \cdot 2\pi$ 的等比数列。

我们选择这种函数形式，是因为我们假设它能够使模型更容易学习基于相对位置的注意力关系。因为对于任意固定偏移量 k ，位置 $pos + k$ 的位置编码 PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。我们也尝试了使用可学习的位置嵌入 [9]，实验结果表明，两种方法产生了几乎相同的结果（见表3中的第（E）行）。我们最终选择正弦形式的位置编码，是因为它可能使模型能够推广到训练过程中未出现过的、更长的序列长度。

4. 为什么选择自注意力机制（Why Self-Attention）

在本节中，我们将自注意力层（self-attention layers）与序列转换任务中常用的循环层（recurrent layers）和卷积层（convolutional layers）进行比较。这些层通常用于将一个长度可变的符号表示序列 $(x_1, \dots, x_n)$ 映射为另一个相同长度的序列 $(z_1, \dots, z_n)$ ，其中 $dx_i, z_i \in R^d$ ，例如典型序列转换模型中的编码器或解码器隐藏层。为了说明采用自注意力机制的原因，我们考虑三个主要目标。

第一个目标是每一层的总体计算复杂度；第二个目标是可并行化计算的程度，即完成计算所需的最少连续操作次数；第三个目标是网络中长距离依赖关系之间的路径长度。

学习长距离依赖关系是许多序列转换任务中的核心挑战。影响模型学习这类依赖关系能力的一个关键因素，是前向和反向信号在网络中需要经过的路径长度。输入序列和输出序列中任意两个位置之间的路径越短，模型学习长距离依赖关系就越容易 [12]。因此，我们也比较了由不同类型层构成的网络中，任意两个输入位置和输出位置之间的最大路径长度。

如表1所示，自注意力层可以通过固定数量的顺序操作连接所有位置，而循环层则需要 $O(n)$ 次顺序操作。从计算复杂度来看，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更加高效，而在机器翻译中最先进模型所使用的句子表示中，这种情况通常较为常见，例如 word-piece [38] 和 byte-pair [31] 表示方法。为了提升处理超长序列任务时的计算效率，可以将自注意力机制限制在输入序列中以输出位置为中心、大小为 rr 的邻域范围内进行计算。然而，这样会使最大路径长度增加到 $O(n/r)$ 。我们计划在未来工作中进一步研究这种方法。

一个卷积核宽度为 $k < n$ 的单个卷积层无法连接所有输入位置和输出位置的组合。若要实现所有位置之间的信息连接，对于连续卷积核，需要堆叠 $O(n/k)$ 个卷积层；对于空洞卷积（dilated convolutions）[18]，则需要 $O(\log_k(n))$ 个卷积层，这会增加网络中任意两个位置之间最长路径的长度。通常情况下，卷积层的计算成本比循环层更高，增加比例约为卷积核大小 k 。不过，可分离卷积（separable convolutions）[6] 可以显著降低计算复杂度，使其达到 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使当 $k = n$ 时，可分离卷积的计算复杂度仍然等同于我们模型中采用的一个自注意力层与一个逐位置前馈层的组合。

此外，自注意力机制还可能带来模型可解释性方面的优势。我们分析了模型中的注意力分布，并在附录中展示和讨论了一些示例。不仅不同的注意力头（attention heads）能够明确学习执行不同的任务，许多注意力头还表现出了与句子的语法结构和语义结构相关的行为。

5. 训练（Training）

本节介绍我们模型的训练过程。

5.1 训练数据和批处理（Training Data and Batching）

我们在标准的 WMT 2014 英德翻译数据集上进行训练，该数据集包含约 450 万个句子对。句子使用 byte-pair encoding (BPE) [3] 进行编码，其中源语言和目标语言共享一个约包含 37000 个 token 的词汇表。对于英法翻译任务，我们使用了规模更大的 WMT 2014 英法数据集，该数据集包含 3600 万个句子，并将 token 划分为一个包含 32000 个 word-piece 的词汇表 [38]。在训练过程中，我们按照近似的序列长度将句子对组合成批次 (batch)。每个训练批次包含若干句子对，其中源语言 token 数量和目标语言 token 数量均约为 25000 个。

5.2 硬件和训练时间安排 (Hardware and Schedule)

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对于本文所述超参数配置下的基础模型 (base model)，每个训练步骤 (training step) 大约需要 0.4 秒。基础模型总共训练了 100000 个训练步骤，耗时约 12 小时。对于大型模型 (big model，如表3最后一行所描述)，每个训练步骤耗时约 1.0 秒。大型模型训练了 300000 个训练步骤，总耗时约 3.5 天。

5.3 优化器 (Optimizer)

我们使用 *Adam* 优化器 [20] 进行训练，其参数设置为 $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ ， $\epsilon = 10^{-9}$

训练过程中，我们根据以下公式动态调整学习率：

$$l_{rate} = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

该学习率策略表示：在前 $warmup_steps$ 个训练步骤中，学习率进行线性增加；之后，学习率按照训练步数的平方根倒数进行衰减。本文设置 $warmup_steps = 4000$

5.4 正则化 (Regularization)

在训练过程中，我们采用了三种正则化方法：

残差 Dropout (Residual Dropout) 我们在每个子层 (sub-layer) 的输出处应用 dropout [33]，然后再将其与子层输入相加，并进行归一化处理。此外，我们还在编码器和解码器堆栈中，将词嵌入 (embeddings) 与位置编码 (positional encodings) 的和输入 dropout。对于基础模型，我们设置 dropout 比例 $P_{drop} = 0.1$

标签平滑 (Label Smoothing) 在训练过程中，我们采用标签平滑 (label smoothing) 方法，设置参数 $\epsilon_{ls} = 0.1$ [36] 标签平滑会降低模型的困惑度 (perplexity) 表现，因为模型会学习表现出更大的不确定性；但是，它能够提升模型的准确率 (accuracy) 和 BLEU 分数。

表2： Transformer 模型在英德翻译和英法翻译的 newstest2014 测试集上，以仅需少量训练成本的情况下，取得了优于此前最先进模型的 BLEU 分数。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

6. 实验结果

6.1 机器翻译 (Machine Translation)

在 WMT 2014 英德翻译任务中，大型 Transformer 模型（表2中的 Transformer(big)）相比之前报道的最佳模型（包括模型集成方法），BLEU 分数提升了超过 2.0 分，达到了新的最优 BLEU 分数 28.4。该模型的具体配置列于表3的最后一行。该模型在 8 块 P100 GPU 上训练了 3.5 天。即使是我们的基础模型（base model），也超过了此前所有已发表的单模型和模型集成方法，同时其训练成本仅为竞争模型的一小部分。

在 WMT 2014 英法翻译任务中，我们的大型模型取得了 41.0 的 BLEU 分数，超过了此前所有已发表的单模型方法，而训练成本不到此前最优模型的四分之一。用于英法翻译的大型 Transformer 模型将 dropout 比例设置为 $P_{drop} = 0.1$ ，而不是 0.3。

对于基础模型，我们使用了一个单独模型，该模型由最后 5 个检查点（checkpoint）的参数平均得到，这些检查点每隔 10 分钟保存一次。对于大型模型，我们平均了最后 20 个检查点。我们采用 beam search 进行解码，其中 beam size 设置为 4，长度惩罚参数设置为 $\alpha = 0.6$ [38]。这些超参数是在开发集（development set）上的实验后确定的。在推理阶段，我们将最大输出长度设置为输入长度加 50，但如果条件允许，则提前终止生成过程 [38]。

表2总结了我们的实验结果，并将翻译质量和训练成本与文献中其他模型架构进行了比较。我们通过以下方式估算模型训练所使用的浮点运算次数：将训练时间、使用的 GPU 数量以及每个 GPU 持续运行时的单精度浮点计算能力估计值相乘。

6.2 模型变体 (Model Variations)

为了评估 Transformer 中不同组件的重要性，我们对基础模型（base model）进行了多种不同方式的调整，并在英德翻译任务的开发集（newstest2013）上测量模型性能变化。我们采用上一节所述的 beam search 方法进行测试，但不使用检查点平均（checkpoint averaging）。实验结果如表3所示。

在表3的 (A) 行中，我们改变了注意力头（attention heads）的数量以及注意力中键和值的维度，同时保持计算量基本不变，具体设置如第3.2.2节所述。实验结果表明，单头注意力（single-head

attention) 的性能比最佳设置低 0.9 BLEU 分；同时，当注意力头数量过多时，模型性能也会下降。

在表3的 (B) 行中，我们观察到，减小注意力键的维度 d_{kd_k} 会降低模型性能。这说明判断查询 (query) 和键 (key) 之间的匹配关系 (compatibility) 并不容易，使用比点积更加复杂的兼容性函数可能会带来更好的效果。

此外，在 (C) 和 (D) 行中，我们进一步观察到，与预期一致，更大的模型通常能够取得更好的性能，而 dropout 对于避免模型过拟合具有明显帮助。

在 (E) 行中，我们将原本使用的正弦位置编码 (sinusoidal positional encoding) 替换为可学习的位置嵌入 (learned positional embeddings) [9]，实验发现两者取得了几乎相同的结果，与基础模型的性能基本一致。

表3：Transformer 架构的不同变体实验。表中未列出的参数值均与基础模型保持一致。所有指标均在英德翻译任务的开发集上进行评估。表中列出的困惑度是基于每个 word-piece 计算得到的，使用的是本文的 byte-pair encoding (BPE) 方法，因此不能与基于每个单词计算得到的困惑度进行比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)		positional embedding instead of sinusoids								4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

6.3 英语成分句法分析 (English Constituency Parsing)

为了评估 Transformer 是否能够推广到其他任务中，我们在英语成分句法分析 (English constituency parsing) 任务上进行了实验。该任务具有一些特殊挑战：输出结果受到严格的结构约束，并且输出序列长度通常明显长于输入序列。此外，在小规模数据条件下，基于 RNN 的序列到序列 (sequence-to-sequence) 模型一直无法达到最先进的性能 [37]。

我们在宾夕法尼亚树库 (Penn Treebank) [25] 的《华尔街日报》 (Wall Street Journal, WSJ) 部分数据集上训练了一个 4 层 Transformer 模型，其中 $d_{\text{model}} = 1024$ ，训练数据约包含 4 万个句子。我

他们还进行了半监督训练实验，使用了更大规模的高置信度数据集（high-confidence corpus）以及 Berkeley Parser 语料库中的数据，总计约包含 1700 万个句子 [37]。

在仅使用 WSJ 数据集的实验中，我们采用了包含 16000 个 token 的词汇表；在半监督实验中，我们采用了包含 32000 个 token 的词汇表。

我们仅在第22节开发集（Section 22 development set）上进行了少量实验，用于选择 dropout 参数（包括注意力 dropout 和残差 dropout，第5.4节）、学习率以及 beam size。除此之外，其他所有参数均保持与英德翻译任务中的基础 Transformer 模型相同。

在推理阶段，我们将最大输出长度设置为输入长度加 300。对于 WSJ 单独训练和半监督训练两种设置，我们均采用 beam size 为 21，并设置长度惩罚参数 $\alpha = 0.3$

表4中的实验结果表明，尽管模型没有针对该任务进行专门调优，我们的模型仍然取得了非常优秀的表现，超过了此前报道的大多数模型，仅次于循环神经网络语法模型（Recurrent Neural Network Grammar） [8]。

与基于 RNN 的序列到序列模型 [37] 不同，即使只使用 WSJ 中 4 万个训练句子进行训练，Transformer 的性能仍然超过了 Berkeley Parser [29]。

表4：Transformer 模型能够很好地推广到英语成分句法分析任务中（实验结果来自 WSJ 数据集的第 23 节测试集）。

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

7. 总结（Conclusion）

在本文中，我们提出了 Transformer，这是第一个完全基于注意力机制（attention）的序列转换模型。该模型使用多头自注意力机制（multi-headed self-attention）替代了编码器-解码器架构中最常使用的循环层（recurrent layers）。

对于机器翻译任务而言，Transformer 的训练速度显著快于基于循环层或卷积层的模型。在 WMT 2014 英德翻译任务和 WMT 2014 英法翻译任务中，Transformer 均取得了新的最优性能。在前一个任务中，我们的最佳模型甚至超过了此前所有已报道的模型集成方法（ensembles）。

我们对基于注意力机制的模型未来发展充满期待，并计划将其应用于更多其他任务。未来，我们希望将 Transformer 扩展到文本之外的其他输入和输出模态任务中，并研究局部受限注意力机制（local, restricted attention mechanisms），以更加高效地处理图像、音频和视频等具有大规模输入和输出的数据。同时，降低生成过程中的序列依赖性（making generation less sequential）也是我们未来研究的目标之一。

我们用于训练和评估模型的代码已经公开，可访问：<https://github.com/tensorflow/tensor2tensor>